

Lista de Exercícios

Funções, Escopo de Variáveis e Passagem de Parâmetros

Orientações gerais

- Todos os programas devem ser escritos em C. Compile sempre com os avisos ativados: `gcc -Wall -Wextra arquivo.c -o programa -lm`.
- Use **exatamente** os protótipos pedidos. Salvo indicação contrária, declare os protótipos antes de `main` e implemente as funções depois de `main`.
- As funções que calculam algo **não** devem ler do teclado nem imprimir na tela: a leitura dos dados e a impressão dos resultados ficam na `main` (exceto quando o objetivo da função for justamente imprimir).
- Não use variáveis globais, a não ser quando o enunciado pedir explicitamente.
- Teste cada função na `main` com vários valores, incluindo casos inválidos ou extremos (zero, negativos, limites).
- Nos exercícios de **teste de mesa**, escreva a resposta no papel **antes** de executar o código; depois execute e compare.
- Os exercícios estão em ordem crescente de dificuldade. Vários reutilizam funções de exercícios anteriores: guarde suas soluções no repositório da disciplina.

Organização da lista

Nível	Exercícios	Conteúdos principais
Básico (40%)	1 a 12	Definição, protótipo e chamada de funções; valor de retorno; funções void; escopo local e global; passagem por valor.
Intermediário (30%)	13 a 21	Passagem por referência (ponteiros); funções que "devolvem" vários valores; vetores como parâmetro; arquivos de cabeçalho; variáveis static; sombreamento de nomes.
Avançado (30%)	22 a 30	Composição de várias funções, bibliotecas próprias, matrizes como parâmetro e aplicações em Matemática (frações, métodos numéricos, polinômios, números complexos).

Parte I — Nível básico

Exercícios 1 a 12: construir, declarar e chamar funções; entender o escopo local e a passagem por valor.

Exercício 1. Minhas primeiras funções

Implemente as funções abaixo e, na `main`, leia os valores do teclado, chame as funções e imprima os resultados.

- `int quadrado(int x);` — retorna x^2 .
- `int cubo(int x);` — retorna x^3 , usando obrigatoriamente a função `quadrado`.

- `double media3(double a, double b, double c);` — retorna a média aritmética dos três valores.

Lembre-se: nenhuma dessas funções deve usar `scanf` ou `printf`.

Exercício 2. Anatomia de um programa com funções

Observe o programa abaixo e responda às perguntas.

```
#include <stdio.h>

double media(double a, double b);    /* (A) */

int main(void) {
    double x = 7.0, y = 8.5, m;
    m = media(x, y);                /* (B) */
    printf("Media = %.2f\n", m);
    return 0;
}

double media(double a, double b) {   /* (C) */
    double resultado;
    resultado = (a + b) / 2.0;
    return resultado;                /* (D) */
}
```

- Qual das linhas marcadas é o protótipo, qual é o cabeçalho da implementação, qual é a chamada e qual é o retorno?
- Quais são os parâmetros formais de `media` e quais são os argumentos usados na chamada?
- Quais variáveis são locais de `main` e quais são locais de `media`? A função `media` pode usar a variável `x`? Por quê?
- O que o compilador informa se a linha (A) for apagada? Teste e copie a mensagem.
- Reescreva o programa sem a linha (A) e sem erros ou avisos de compilação (dica: mude a posição da implementação).

Exercício 3. Par ou ímpar

Implemente `int ehPar(int n)`, que retorna 1 se `n` for par e 0 caso contrário. Usando essa função, escreva uma `main` que leia um inteiro `N` e imprima todos os números pares entre 1 e `N` e, ao final, a quantidade de números ímpares nesse intervalo. A `main` não pode usar o operador `%` diretamente.

Exercício 4. O maior de todos

Implemente `int maior2(int a, int b)`; e, em seguida, `int maior3(int a, int b, int c)`. A função `maior3` deve ter uma única instrução (`return ...;`) que use apenas chamadas a `maior2`, sem nenhum `if`. Faça o mesmo para `int menor2(int a, int b)`; e `int menor3(int a, int b, int c)`. Na `main`, leia três números e imprima o maior e o menor deles.

Exercício 5. Tabela de conversão de temperaturas

Implemente `double celsiusParaFahrenheit(double c)`; (use $F = 9C/5 + 32$) e `double fahrenheitParaCelsius(double f)`. Na `main`, imprima uma tabela de -10°C a 40°C , de 5 em 5 graus, com três colunas: a temperatura em $^{\circ}\text{C}$, o valor convertido para $^{\circ}\text{F}$ e o valor em $^{\circ}\text{F}$ convertido de volta para $^{\circ}\text{C}$. A terceira coluna deve ser igual à primeira: por que isso confirma que uma função é a inversa da outra?

Exercício 6. Procedimentos: funções do tipo void

Implemente `void repete(char c, int n)`, que imprime o caractere `c` `n` vezes seguidas, **sem** quebrar a linha. Usando apenas chamadas a `repete` e `printf("\n")`, implemente `void retangulo(int largura, int altura)`, que desenha um retângulo com borda de asteriscos e interior vazio. Exemplo para largura 6 e altura 4:

```
*****
*      *
*      *
*      *
*****
```

Por que as funções `repete` e `retangulo` não precisam de um `return` com valor? O que aconteceria se a `main` tentasse fazer `int r = retangulo(6, 4);`?

Exercício 7. Fatorial com validação

Implemente a versão iterativa de `long long int fatorial(int n)`. A função deve retornar `-1` quando `n < 0` ou `n > 20` (valores inválidos). Na `main`, imprima o fatorial de todos os inteiros de 0 a 22; quando a função retornar `-1`, a `main` deve imprimir uma mensagem de erro (a função não imprime nada). Pesquise a constante `LLONG_MAX` da biblioteca `limits.h` e explique por que 20 é o maior valor aceito.

Exercício 8. Organizando o código: protótipos e implementações

Implemente as funções `double areaCirculo(double r)`, `double perimetroCirculo(double r)`; e `double volumeEsfera(double r)`. Se `r` for negativo, todas devem retornar `-1`. Use a constante `M_PI` da biblioteca `math.h`.

- Versão 1: declare e implemente as três funções ao mesmo tempo, **antes** da `main`.
- Versão 2: declare os três protótipos no início do arquivo e implemente as funções **depois** da `main`.
- Versão 3: apague os protótipos da versão 2 e compile. Anote as mensagens do compilador e explique-as com base no que foi visto em aula sobre protótipos.

Exercício 9. Teste de mesa: escopo local e passagem por valor

```
#include <stdio.h>

void altera(int x) {
    x = x + 10;
    printf("Dentro de altera: x = %d\n", x);
}

int main(void) {
    int x = 5;
    printf("Antes: x = %d\n", x);
    altera(x);
    printf("Depois: x = %d\n", x);
    return 0;
}
```

- Sem executar, escreva a saída exata do programa.
- A variável `x` da `main` e o parâmetro `x` de `altera` são a mesma variável? Justifique usando os conceitos de escopo e de passagem por valor.
- Execute o programa e confira sua resposta.

Exercício 10. Teste de mesa: variáveis globais e escopo de instrução

```
#include <stdio.h>

int contador = 0;

void registra(void) {
    contador++;
    printf("registra: contador = %d\n", contador);
}

int main(void) {
    for (int i = 0; i < 3; i++) {
        registra();
    }
    printf("main: contador = %d\n", contador);
    printf("main: i = %d\n", i);
    return 0;
}
```

- Uma das linhas impede a compilação do programa. Qual é essa linha e por quê?
- Apague essa linha e escreva, sem executar, a saída do programa.
- Por que `registra` consegue alterar `contador` sem receber nenhum parâmetro?
- Cite uma desvantagem de usar variáveis globais em programas grandes.

Exercício 11. Retornar ou alterar?

Considere as duas funções abaixo:

```
void dobraV(int x) { x = 2 * x; }
int  dobraR(int x) { return 2 * x; }
```

- Na `main`, declare `int a = 7`, chame `dobraV(a)`; e imprima `a`. Em seguida, faça `a = dobraR(a)`; e imprima `a` novamente. Explique a diferença entre os resultados.
- Implemente `int potencia(int base, int exp)`; (com $\text{exp} \geq 0$) usando um laço que **decrementa o próprio parâmetro `exp`** até zero. Na `main`, imprima os valores das variáveis usadas como argumentos antes e depois da chamada e explique por que elas não mudam.

Exercício 12. Brincando com os dígitos

Implemente as funções abaixo, considerando o valor absoluto de `n` quando ele for negativo:

- `int contaDigitos(int n)`; — quantidade de dígitos de `n` (por exemplo, 3 para 507);
- `int somaDigitos(int n)`; — soma dos dígitos de `n` (por exemplo, 12 para 507);
- `int inverteNumero(int n)`; — número com os dígitos na ordem inversa (por exemplo, 4321 para 1234);
- `int ehPalindromo(int n)`; — retorna 1 se `n` for palíndromo, usando obrigatoriamente `inverteNumero`.

Na `main`, imprima todos os palíndromos entre 1 e 1000, junto com a quantidade e a soma de seus dígitos.

Parte II — Nível intermediário

Exercícios 13 a 21: passagem por referência, vetores como parâmetros, bibliotecas próprias e regras de escopo.

Exercício 13. Troca de valores

- Implemente `void trocaErrada(int a, int b);` e `void troca(int *a, int *b);`, ambas tentando trocar os valores recebidos.
- Na `main`, imprima os valores de duas variáveis `x` e `y` antes e depois de chamar cada uma das funções.
- Para cada função, desenhe um esquema da memória (caixas com os nomes, endereços e valores das variáveis) mostrando o que é copiado para a função no momento da chamada.
- Por que, na chamada de `troca`, é preciso escrever `&x` e `&y`?

Exercício 14. Ordenando duplas e trios

Implemente `void ordena2(int *a, int *b);` depois da chamada, `*a` deve conter o maior dos dois valores e `*b` o menor. Em seguida, implemente `void ordena3(int *a, int *b, int *c);`, que deixa os três valores em ordem decrescente usando **apenas** chamadas a `ordena2` (sem nenhuma comparação direta). Quantas chamadas a `ordena2` são necessárias? Teste com as seis permutações de 1, 2 e 3.

Exercício 15. Uma divisão, dois resultados

Implemente `int divide(int a, int b, int *q, int *r);`, que calcula o quociente e o resto da divisão inteira de `a` por `b` **usando apenas subtrações sucessivas** (sem `/` e sem `%`). A função retorna 1 em caso de sucesso e 0 se `a < 0` ou `b ≤ 0`; nesse caso, `*q` e `*r` não devem ser alterados. Na `main`, confira os resultados comparando com os operadores `/` e `%`. Por que é uma boa prática usar o valor de retorno para indicar sucesso ou erro e os ponteiros para devolver os resultados?

Exercício 16. Estatísticas de um vetor

Retome o problema visto em aula e implemente `void estatisticas(const double x[], int n, double *max, double *min, double *media);`. Na `main`, leia `n` ($1 \leq n \leq 100$) e os `n` valores, chame a função e imprima os três resultados. Em seguida, implemente `double desvioPadrao(const double x[], int n, double media);` e use-a com a média calculada. Explique o papel da palavra `const` no parâmetro `x`.

Exercício 17. Vetores são sempre passados por referência

Implemente as funções:

- `void preencheAleatorio(int v[], int n, int a, int b);` — preenche o vetor com inteiros aleatórios no intervalo `[a, b]` (use `rand`; a chamada a `srand` fica na `main`);
 - `void imprimeVetor(const int v[], int n);`
 - `void inverteVetor(int v[], int n);` — inverte a ordem dos elementos, usando a função `troca` do exercício 13;
 - `int contaOcorrencias(const int v[], int n, int valor);`
- Na `main`, preencha um vetor de 15 posições, imprima-o, inverta-o, imprima-o de novo e informe quantas vezes aparece um valor lido do teclado.
 - Explique por que `inverteVetor` altera o vetor da `main` mesmo sem usar `&` na chamada.

- c) Tente alterar `v[0]` dentro de `imprimeVetor`. O que o compilador informa? Por quê?

Exercício 18. Minha primeira biblioteca de funções

Organize o código em três arquivos: `aritmetica.h` (somente os protótipos), `aritmetica.c` (as implementações) e `main.c` (o programa principal, que inclui a biblioteca com `#include "aritmetica.h"`). A biblioteca deve conter:

- `int mdc(int a, int b);` — máximo divisor comum pelo algoritmo de Euclides;
- `int mmc(int a, int b);` — mínimo múltiplo comum, usando `mdc`;
- `int ehPrimo(int n);`
- `int primosEntreSi(int a, int b);` — usando `mdc`.

A `main` deve ler pares de números até que o usuário digite 0 0, imprimindo o mdc, o mmc e se eles são primos entre si; ao final, imprime todos os primos menores que 100. Compile com `gcc -Wall main.c aritmetica.c -o aritmetica`. O que acontece se você compilar apenas `main.c`? Pesquise o que são "include guards" (`#ifndef ... #define ... #endif`) e acrescente-os ao arquivo `.h`.

Exercício 19. Equação do segundo grau

Implemente `int raizes(double a, double b, double c, double *x1, double *x2)`, que resolve $ax^2 + bx + c = 0$ e retorna: `-1` se `a = 0` (a equação não é do segundo grau); `0` se $\Delta < 0$ (sem raízes reais); `1` se $\Delta = 0$ (raiz dupla, com `*x1 = *x2`); `2` se $\Delta > 0$ (com `*x1 < *x2`). Na `main`, use um `switch` sobre o valor retornado para imprimir a mensagem adequada. Teste pelo menos com os coeficientes (1, -3, 2), (1, 2, 1), (1, 0, 1) e (0, 2, 1).

Exercício 20. Variáveis locais estáticas

```
#include <stdio.h>

int contaA(void) { int c = 0; c++; return c; }
int contaB(void) { static int c = 0; c++; return c; }

int main(void) {
    for (int i = 1; i <= 3; i++) {
        int a = contaA();
        int b = contaB();
        printf("%d: A = %d, B = %d\n", i, a, b);
    }
    return 0;
}
```

- Sem executar, escreva a saída do programa.
- Pesquise o efeito da palavra-chave `static` em uma variável local e explique a diferença entre a variável `c` de `contaA` e a de `contaB`, considerando o escopo e o tempo de vida de cada uma.
- Implemente `int proximoId(void)`, que retorna 1 na primeira chamada, 2 na segunda, 3 na terceira e assim por diante, sem usar variáveis globais.
- Que vantagem essa solução tem em relação ao uso de uma variável global (compare com o exercício 10)?

Exercício 21. Sombreamento de nomes

```
#include <stdio.h>

int x = 100;
```

```

void mostra(void) {
    printf("mostra: x = %d\n", x);
}

int main(void) {
    printf("1: x = %d\n", x);
    int x = 1;
    printf("2: x = %d\n", x);
    if (x > 0) {
        int x = 2;
        x++;
        printf("3: x = %d\n", x);
    }
    printf("4: x = %d\n", x);
    for (int x = 10; x < 12; x++) {
        printf("5: x = %d\n", x);
    }
    mostra();
    printf("6: x = %d\n", x);
    return 0;
}

```

- Sem executar, escreva a saída exata do programa.
- Para cada `printf`, indique a qual variável `x` ele se refere: a global, a local da `main`, a do bloco do `if` ou a do escopo de instrução do `for`.
- Quantas variáveis distintas chamadas `x` existem no programa? Onde começa e onde termina o escopo de cada uma?
- Reescreva o programa com nomes diferentes para cada variável, mantendo exatamente a mesma saída. Por que essa versão é mais fácil de ler?

Parte III — Nível avançado

Exercícios 22 a 30: combinar várias funções para resolver problemas maiores, com aplicações em Matemática.

Exercício 22. Aritmética de frações

Uma fração será representada por dois inteiros: numerador e denominador. Implemente:

- `int simplifica(int *num, int *den);` — reduz a fração à forma irredutível, com denominador positivo (por exemplo, $-4/-6$ vira $2/3$ e $3/-9$ vira $-1/3$). Retorna 0 se `*den` for zero. Use a função `mdc` do exercício 18;
- `int somaFracoes(int n1, int d1, int n2, int d2, int *nr, int *dr);`
- `int multiplicaFracoes(int n1, int d1, int n2, int d2, int *nr, int *dr);`
- `int divideFracoes(int n1, int d1, int n2, int d2, int *nr, int *dr);` — retorna 0 se a segunda fração for nula;
- `void imprimeFracao(int num, int den);` — imprime, por exemplo, $3/4$ (ou apenas 5, quando o denominador for 1).

Todas as operações devem devolver o resultado já simplificado. Na `main`, calcule os números harmônicos $H_n = 1 + 1/2 + 1/3 + \dots + 1/n$, na forma de fração irredutível, para $n = 1, 2, \dots, 15$. Depois, tente ir até $n = 25$: a partir de que valor os resultados ficam errados? Explique por quê.

Exercício 23. Raiz de uma função pelo método da bisseção

Implemente `double f(double x)`, que retorna $x^3 - x - 2$, e a função

```
int bissecao(double a, double b, double tol, int maxIter,
            double *raiz, int *iter);
```

Ela deve retornar 0 se $f(a) \cdot f(b) > 0$ (sem garantia de raiz no intervalo), 1 se convergiu (quando $(b - a)/2 < \text{tol}$) e 2 se atingiu `maxIter` iterações sem convergir. Em `*raiz` e `*iter` ficam a aproximação obtida e o número de iterações realizadas.

- Na `main`, aplique o método no intervalo $[1, 2]$ com tolerâncias 10^{-3} , 10^{-6} e 10^{-10} , imprimindo a raiz, o número de iterações e o valor de f na raiz.
- Implemente `int iteracoesTeoricas(double a, double b, double tol)`, que calcula o menor n tal que $(b - a)/2^n < \text{tol}$, e compare com o número de iterações obtido no item anterior.
- Modifique apenas a função `f` para calcular $\cos(x) - x$ e encontre sua raiz em $[0, 1]$. Por que nenhuma outra parte do programa precisou mudar?

Exercício 24. Integração numérica

Seja `double g(double x)`; uma função que retorna $\sin(x)$. Implemente:

- `double trapezio(double a, double b, int n)`; — regra dos trapézios composta com n subintervalos;
- `int simpson(double a, double b, int n, double *resultado)`; — regra de Simpson $1/3$ composta; retorna 0 se n não for par e positivo.

Na `main`, sabendo que a integral de $\sin(x)$ de 0 a π vale exatamente 2, imprima uma tabela com $n = 2, 4, 8, \dots, 1024$ contendo, para cada método, o valor aproximado, o erro absoluto e a razão entre o erro atual e o erro da linha anterior. O que você observa nas razões de cada método (aproximadamente 4 e 16)? Como isso se relaciona com a ordem de cada método?

Desafio: pesquise sobre ponteiros para funções e reescreva `trapezio` com o protótipo `double trapezio(double (*func)(double), double a, double b, int n)`, de modo que a mesma função possa integrar $\sin(x)$, e^x ou x^2 .

Exercício 25. Matrizes como parâmetros

Use `#define MAX 10` e matrizes quadradas $n \times n$ ($n \leq \text{MAX}$). Implemente:

- `void leMatriz(int n, double A[][MAX])`; e `void imprimeMatriz(int n, double A[][MAX])`;
 - `void multiplica(int n, double A[][MAX], double B[][MAX], double C[][MAX])`; — $C = AB$;
 - `void transposta(int n, double A[][MAX], double T[][MAX])`;
 - `double traco(int n, double A[][MAX])`; e `int ehSimetrica(int n, double A[][MAX])`;
 - `int iguais(int n, double A[][MAX], double B[][MAX], double tol)`; — retorna 1 se todos os elementos diferirem em menos de tol ;
 - `void potenciaMatriz(int n, double A[][MAX], int k, double P[][MAX])`; — $P = A^k$, usando `multiplica`.
- Na `main`, leia duas matrizes A e B e verifique numericamente que $\text{tr}(AB) = \text{tr}(BA)$ e que $(AB)^T = B^T A^T$.
 - Verifique que $A \cdot A^T$ é sempre simétrica.

- c) Por que não é possível chamar `multiplica(n, A, B, A)`; (usar a mesma matriz como entrada e saída) sem estragar o resultado? Como isso influencia a implementação de `potenciaMatriz`?
- d) Por que, ao contrário dos vetores, é obrigatório informar o tamanho da segunda dimensão (`MAX`) no parâmetro?

Exercício 26. Crivo de Eratóstenes e a conjectura de Goldbach

Implemente:

- `int crivo(int n, int ehPrimo[])`; — preenche `ehPrimo[i]` com 1 se *i* for primo e 0 caso contrário, para $0 \leq i \leq n$, pelo crivo de Eratóstenes; retorna a quantidade de primos encontrados;
- `int goldbach(int n, const int ehPrimo[], int *p, int *q)`; — para *n* par maior que 2, encontra primos $p \leq q$ com $p + q = n$, com o menor *p* possível; retorna 0 se *n* for inválido ou se a decomposição não existir;
- `int contaDecomposicoes(int n, const int ehPrimo[])`; — quantidade de pares (*p*, *q*), com $p \leq q$, tais que $p + q = n$.

Na `main`, execute o crivo uma única vez até 10000 e: (a) imprima as decomposições de todos os pares de 4 a 100; (b) verifique a conjectura para todos os pares até 10000; (c) encontre, até 10000, o número par com mais decomposições. Por que o vetor `ehPrimo` é passado como `const` para `goldbach` e `contaDecomposicoes`, mas não para `crivo`?

Exercício 27. Mudança de base

Implemente as funções auxiliares `int valorDigito(char c)`; (converte '0'–'9' e 'A'–'F' em 0–15, retornando -1 para caracteres inválidos) e `char caractereDigito(int d)`; (operação inversa).

Usando-as, implemente:

- `int decimalParaBase(int n, int base, char s[])`; — escreve em *s* a representação de $n \geq 0$ na base indicada ($2 \leq \text{base} \leq 16$), como uma string terminada em '\0'; retorna o número de dígitos ou 0 se os dados forem inválidos;
- `int baseParaDecimal(const char s[], int base, int *valor)`; — converte a string para decimal; retorna 0 se algum caractere não for um dígito válido naquela base.

Na `main`, imprima uma tabela dos números de 0 a 64 nas bases 2, 8 e 16 e verifique, para todo *n* de 0 a 10000 e todas as bases de 2 a 16, que converter para a base e de volta para decimal devolve o valor original. Explique por que *s* pode ser alterada dentro de `decimalParaBase` e o resultado aparece na `main`.

Exercício 28. Polinômios e o método de Newton

Um polinômio $p(x) = c_0 + c_1x + c_2x^2 + \dots + c_nx^n$ será representado pelo vetor de coeficientes `c[0..grau]`. Implemente:

- `double horner(const double c[], int grau, double x)`; — avalia $p(x)$ pelo método de Horner;
- `int derivada(const double c[], int grau, double d[])`; — escreve em *d* os coeficientes de $p'(x)$ e retorna o grau da derivada;
- `int newton(const double c[], int grau, double x0, double tol, int maxIter, double *raiz, int *iter)`; — método de Newton, usando `horner` e `derivada`; retorna 1 se convergiu, 0 se a derivada se anulou e 2 se atingiu `maxIter`.

Na **main**, use $p(x) = x^3 - 2x^2 - 5x + 6$, cujas raízes são -2 , 1 e 3 . Aplique o método a partir de $x_0 = -4$, 0 , $1,5$, $2,5$ e 5 e mostre para qual raiz cada chute inicial converge e com quantas iterações. Compare o número de iterações com o da bisseção (exercício 23) para a mesma tolerância.

Exercício 29. Teste de mesa integrador

```
#include <stdio.h>

int g = 10;

void f1(int g) {
    g = g * 2;
    printf("f1: g = %d\n", g);
}

void f2(int *p) {
    *p = *p + g;
    g++;
    printf("f2: *p = %d, g = %d\n", *p, g);
}

int f3(void) {
    static int s = 0;
    int t = 0;
    s += 2;
    t += 2;
    return s + t;
}

void f4(int v[], int n) {
    for (int i = 0; i < n; i++)
        v[i] += i;
    n = 0;
}

int main(void) {
    int g = 1, n = 3;
    int v[3] = {5, 5, 5};
    f1(g);
    printf("main: g = %d\n", g);
    f2(&g);
    printf("main: g = %d\n", g);
    printf("f3: %d\n", f3());
    printf("f3: %d\n", f3());
    f4(v, n);
    printf("v = %d %d %d, n = %d\n", v[0], v[1], v[2], n);
    return 0;
}
```

- Sem executar, escreva a saída exata do programa.
- Para cada ocorrência do nome **g** nas funções **f1**, **f2** e **main**, indique se ela se refere à variável global, a um parâmetro ou a uma variável local.
- Desenhe a pilha de chamadas e os valores de todas as variáveis visíveis no momento em que o **printf** de **f2** é executado.
- Por que, após a chamada de **f4**, o vetor **v** foi alterado, mas a variável **n** da **main** continua valendo 3?
- Execute o programa e confira suas respostas. Para cada diferença encontrada, explique qual conceito foi aplicado de forma incorreta.

Exercício 30. Projeto: biblioteca de números complexos

Sem usar `struct` (conteúdo que ainda será estudado), um número complexo $z = a + bi$ será representado por dois `double`: a parte real e a parte imaginária. Crie a biblioteca `complexos.h` / `complexos.c` com as funções:

- `void somaC(double a, double b, double c, double d, double *re, double *im);`
- `void multiplicaC(double a, double b, double c, double d, double *re, double *im);`
- `int divideC(double a, double b, double c, double d, double *re, double *im);` — retorna 0 se $c + di = 0$;
- `double moduloC(double a, double b);` e `double argumentoC(double a, double b);` (use `atan2`);
- `void polarParaRetangular(double r, double theta, double *a, double *b);`
- `void potenciaC(double a, double b, int n, double *re, double *im);` — z^n pela fórmula de De Moivre;
- `int raizesC(double a, double b, int n, double re[], double im[]);` — armazena nos vetores as n raízes n -ésimas de z e retorna n (ou 0 se $n \leq 0$);
- `void imprimeC(double a, double b);` — imprime, por exemplo, $3.00 - 2.00i$.

Escreva uma `main` com um menu (usando `switch`) que permita escolher a operação. Inclua uma opção de autoteste que verifique: $(1 + i)^8 = 16$; as raízes cúbicas de 8 são 2 e $-1 \pm \sqrt{3}i$; cada raiz calculada por `raizesC`, elevada a n com `potenciaC`, devolve z com erro menor que 10^{-9} ; e `potenciaC` coincide com o resultado de multiplicar z por ele mesmo n vezes usando `multiplicaC`.